CENTRO UNIVERSITÁRIO DAS FACULDADES ASSOCIADAS DE ENSINO - FAE
UNIFAE
Largo Engenheiro Paulo de Almeida Sandeville, 15 – Caixa Postal 96
Tel./fax (0xx19) 3623-3022, CEP: 13.870-377 - São João da Boa Vista/SP
HOME PAGE: www.fae.br – E-mail: secretaria@fae.br

APÊNDICE E - DETALHAMENTO DA ANÁLISE DE MODULARIDADE E ESCALABILIDADE

João Ferraz Netto[1]
Gleisson Otávio Maldonado Valentim[2]
Rogério Colpani[3]

1 Comparação quanto à modularidade

Tabela 1 – Principais componentes reutilizáveis da BT.

Nº	Componente	Descrição / Função na BT
1	*BT_Node*	Classe base abstrata utilizada pelos diferentes tipos de nós da árvore.
2	*BT_Selector*	Estrutura de controle que organiza comportamentos de acordo com uma ordem de prioridade.
3	*BT_Sequence*	Estrutura de controle que executa condições e ações de forma sequencial.
4	*BT_Inverter*	Decorador que inverte o resultado retornado por um nó filho.
5	*BT_Condition*	Nó folha que permite transformar uma condição `Func<bool>` em um nó reutilizável.
6	*BT_Action*	Nó folha que permite transformar uma ação `Func<NodeStatus>` em um nó reutilizável.
7	*CanSeePlayer()*	Módulo de percepção visual reutilizado como condição em nós da árvore.
8	*CanHearPlayer()*	Módulo de percepção auditiva reutilizado como condição em nós da árvore.
9	*CanSeePlayerLight()*	Módulo de percepção luminosa reutilizado como condição em nós da árvore.

Fonte: Elaborado pelos autores (2026).

[1] Graduando em Engenharia de *Software* pela UNIFAE – joao.netto@sou.fae.br.
[2] Graduando em Engenharia de *Software* pela UNIFAE – gleisson.valentim@sou.fae.br.
[3] Professor do Curso de Engenharia de *Software* da UNIFAE - rogerio.colpani@prof.fae.br.

Entre os componentes reutilizáveis identificados na implementação baseada na BT, destacam-se as estruturas utilizadas para organizar e executar os diferentes elementos da árvore, além dos módulos sensoriais de percepção do NPC. Esses componentes são apresentados na Tabela 1.

Conforme apresentado na Tabela 1, os componentes possuem responsabilidades específicas dentro da estrutura da árvore. O *BT_Node* funciona como classe base para os nós, o *BT_Selector* e o *BT_Sequence* organizam o fluxo de controle por prioridade e sequência, o *BT_Inverter* altera o resultado de nós filhos, e os componentes *BT_Condition* e BT_Action encapsulam condições e ações em estruturas reutilizáveis. Somam-se a eles os módulos de percepção sensorial (*CanSeePlayer*(), *CanHearPlayer*() e *CanSeePlayerLight*()), que operam como condições modulares desacopladas da lógica de decisão, totalizando os 9 componentes reutilizáveis da arquitetura.

Esses componentes contribuem para a modularidade da BT porque permitem construir diferentes comportamentos a partir de estruturas já existentes, sem exigir alterações na estrutura principal da árvore para cada nova combinação de condições e ações.

Na FSM, também foram identificadas funções reutilizáveis relacionadas à percepção do jogador e ao controle dos comportamentos do NPC. Entre elas estão *CanSeePlayer*(), responsável por verificar a detecção visual do jogador; *CanHearPlayer*(), utilizada para verificar a detecção sonora; *CanSeePlayerLight*(), responsável pela detecção da luz da lanterna; *FindNewPatrolDestination*(), utilizada para definir novos destinos de patrulhamento; e *SwitchState*(), responsável por gerenciar a transição, desativação do estado anterior e inicialização do novo estado ativo.

Essas funções concentram operações específicas que podem ser chamadas por diferentes estados, evitando a duplicação da mesma lógica no código. Assim, a reutilização na FSM ocorre principalmente por meio do encapsulamento dessas operações em funções independentes, enquanto na BT ocorre pela composição de nós reutilizáveis. Essa diferença permite comparar não apenas a quantidade de componentes reutilizáveis, mas também a forma como cada arquitetura organiza e reaproveita sua lógica.

Além da reutilização de componentes, foi analisada a quantidade de pontos que precisaram ser modificados para a inclusão de um novo comportamento. Para

essa análise, foi considerado um comportamento de fuga, aplicado conceitualmente aos dois algoritmos. Na FSM, foram identificados sete pontos de alteração, apresentados na Tabela 2.

Tabela 2 – Pontos de alteração na FSM (Inclusão do comportamento de fuga).

Nº	Ponto de alteração	O que seria alterado
1	Declaração do novo estado	Adicionar o estado *Fleeing* ao conjunto enumerador *AIState*
2	Método de entrada do estado	Adicionar a inicialização do novo estado no método *OnEnterState()*
3	Método de saída do estado	Adicionar a rotina de desativação/limpeza no método *OnExitState()*
4	Execução do comportamento	Adicionar a lógica contínua de fuga no método *ExecutarEstadoAtual()*
5	Avaliação das transições	Adicionar as condições de entrada/saída de fuga no método central *AvaliarTransicoes()*
6	Transições provenientes de outros estados	Revisar os estados existentes em *AvaliarTransicoes()* para permitir a transição para fuga
7	Estrutura de transições	Atualizar as relações condicionais necessárias para integrar o novo estado à malha da FSM

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 2, a inclusão do comportamento de fuga na FSM exigiria alterações em diferentes partes da implementação. Além da criação do novo estado, seria necessário definir seu ciclo de entrada, execução e saída, bem como estabelecer as condições responsáveis pela transição para esse estado. Também seria necessário revisar os estados existentes e atualizar suas relações de transição para integrar o novo comportamento. Dessa forma, a inclusão da fuga resultaria em sete pontos de alteração distribuídos pela estrutura da FSM.

Na implementação baseada na BT, a inclusão do mesmo comportamento foi analisada considerando a criação dos elementos necessários e sua integração à árvore. Os pontos de alteração identificados são apresentados na Tabela 3.

Tabela 3 – Pontos de alteração na BT (Inclusão do comportamento de fuga).

Nº	Ponto de alteração	O que seria alterado
1	Condição de fuga	Criar um nó Condition para verificar se a fuga deve ser iniciada
2	Ação de fuga	Criar um nó Action responsável por executar o comportamento
3	Conexão do novo ramo	Adicionar o novo ramo à estrutura da árvore, respeitando sua prioridade

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 3, a inclusão do comportamento de fuga na BT seria concentrada em três pontos principais. Primeiro, seria criada uma condição responsável por verificar se o comportamento deve ser iniciado. Em seguida, seria criada a ação responsável por executar a fuga. Por fim, o novo ramo seria conectado à árvore de acordo com sua prioridade de execução. Dessa forma, o novo comportamento poderia ser integrado sem a necessidade de modificar diversos comportamentos já existentes.

2 Comparação quanto à escalabilidade

Na FSM, a implementação inicial apresentava 4 estados e 9 transições (considerando as transições condicionais da lógica de patrulha e perseguição e as transições globais de recuperação e colisão). Após a incorporação dos comportamentos de investigação sensorial (som e lanterna) e a formalização da busca em esconderijos como estado dedicado, a implementação final passou a apresentar 6 estados e 17 transições.

A evolução estrutural ocorreu pela inclusão dos estados de investigação e busca em esconderijos (que na versão inicial operava como uma rotina temporizada dentro da perseguição) e pela criação das relações necessárias entre esses estados e os comportamentos já existentes.

a FSM inicial, os quatro estados eram: patrulhamento, espera em ponto de patrulha, perseguição e ataque. Para detalhar como esses comportamentos estavam relacionados, a Tabela 4 apresenta as 9 transições identificadas na estrutura inicial.

Tabela 4 – Principais transições da FSM inicial.

Nº	Origem	Destino	Condição
1	Patrulhamento	Perseguição	Jogador visualizado (*CanSeePlayer*)
2	Patrulhamento	Espera em ponto de patrulha	Chegada ao destino de patrulha
3	Espera em ponto de patrulha	Perseguição	Jogador visualizado (*CanSeePlayer*)
4	Espera em ponto de patrulha	Patrulhamento	Término do tempo de espera
5	Perseguição	Ataque	Jogador alcança a distância de captura
6	Perseguição	Patrulhamento	Perda de contato visual prolongada
7	Perseguição	Patrulhamento	Investigação de esconderijo concluída sem encontrar o jogador
8	Patrulhamento / Espera	Patrulhamento	Recuperação por teleporte (distância do jogador > 60m)
9	Patrulhamento / Espera / Perseguição	Ataque	Colisão física imediata com o jogador (*OnTriggerEnter*)

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 4, a versão inicial da FSM organizava o comportamento do NPC principalmente entre patrulhamento, espera, perseguição e ataque. O jogador poderia ser identificado durante o patrulhamento ou a espera, provocando a entrada na perseguição. Durante a perseguição, o alcance do jogador conduzia ao ataque, enquanto sua perda fazia o NPC retornar ao patrulhamento. A

chegada ao ponto de patrulha provocava a mudança para o estado de espera, que posteriormente retornava ao patrulhamento.

Além dessas, a rotina de investigação em esconderijo, executada como corrotina, conduzia o NPC de volta ao patrulhamento, o teleporte de recuperação reposicionava o monstro em patrulha caso ficasse muito distante, e a colisão física imediata garantia a transição para ataque.

Com a expansão do sistema, foram incorporados os estados formais de investigação e busca em esconderijos, além das respectivas relações condicionais com os estados existentes. Às 17 transições da implementação final são apresentadas na Tabela 5.

Tabela 5 – Principais transições da implementação final da FSM.

Nº	Origem	Destino	Condição
1	Patrulhamento	Perseguição	Jogador visualizado (*CanSeePlayer*)
2	Patrulhamento	Investigação	Som de corrida captado (*CanHearPlayer*)
3	Patrulhamento	Investigação	Luz da lanterna detectada (*CanSeePlayerLight*)
4	Patrulhamento	Espera	Chegada ao destino de patrulha
5	Espera	Perseguição	Jogador visualizado (*CanSeePlayer*)
6	Espera	Investigação	Som de corrida captado (*CanHearPlayer*)
7	Espera	Investigação	Luz da lanterna detectada (*CanSeePlayerLight*)
8	Espera	Patrulhamento	Término do tempo de espera
9	Investigação	Perseguição	Jogador visualizado (*CanSeePlayer*)
10	Investigação	Patrulhamento	Investigação encerrada sem encontrar o jogador

Nº	Origem	Destino	Condição
11	Perseguição	Ataque	Jogador alcançado em distância de captura
12	Perseguição	Busca em esconderijos	Jogador se escondeu nas proximidades sem ser visto
13	Perseguição	Patrulhamento	Perda de contato visual prolongada
14	Busca em esconderijos	Perseguição	Luz da lanterna detectada no esconderijo
15	Busca em esconderijos	Patrulhamento	Tempo de varredura encerrado
16	Patrulhamento / Espera / Investigação / Busca	Patrulhamento	Recuperação por teleporte anti-perda (distância > 60m)
17	Qualquer estado (exceto Ataque)	Ataque	Colisão física imediata com o jogador (*OnTriggerEnter*)

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 5, a expansão da FSM não ocorreu somente pela criação de novos estados, mas também pelo estabelecimento de novas transições entre os comportamentos.

Durante o patrulhamento e a espera, o NPC pode iniciar uma perseguição quando visualiza o jogador ou iniciar uma investigação quando identifica um som ou a luz da lanterna. Durante a investigação, a localização do jogador permite a mudança para a perseguição, enquanto o encerramento da investigação sem encontrá-lo faz com que o NPC retorne ao patrulhamento.

Na perseguição, o alcance do jogador provoca a transição para o ataque, enquanto a identificação de um esconderijo próximo inicia a busca. Caso o jogador seja perdido, o NPC retorna ao patrulhamento.

Durante a busca em esconderijos, a detecção da lanterna permite retomar a perseguição, enquanto o encerramento da busca conduz novamente ao patrulhamento. Somam-se a essas o teleporte de recuperação e o gatilho de colisão física imediata. Dessa forma, as 17 transições da versão final representam as

relações condicionais e os mecanismos de segurança necessários para conectar os seis estados da implementação.

Na BT, a estrutura inicial apresentava 14 nós. Para demonstrar como esses nós estavam organizados antes da incorporação dos novos comportamentos, a Tabela 6 apresenta os nós identificados na implementação inicial.

Tabela 6 – Nós da implementação inicial baseada em BT.

Nº	Tipo	Função / Comportamento
1	*Selector*	Raiz da árvore
2	*Sequence*	Ramo de Ataque
3	*Condition*	*IsAttacking*
4	*Action*	*ExecuteAttack*
5	*Sequence*	Ramo de Investigação de esconderijo
6	*Condition*	*IsInvestigating* (esconderijo)
7	*Action*	*ExecuteInvestigation* (esconderijo)
8	*Sequence*	Ramo de Perseguição
9	*Condition*	*ShouldChase*
10	*Action*	*ExecuteChase*
11	*Sequence*	Ramo de Espera
12	*Condition*	*IsWaitingAtPoint*
13	*Action*	*ExecuteWait*
14	*Action*	Ramo de Patrulha (*ExecutePatrol*)

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 6, a estrutura inicial da BT era composta por um nó raiz do tipo *selector* e por sequências destinadas aos comportamentos de

ataque, investigação de esconderijo, perseguição e espera, além da ação de patrulhamento. As condições determinavam quando cada comportamento poderia ser executado, enquanto as ações realizavam as operações correspondentes. Na árvore, às 13 ligações conectam a raiz aos ramos e estes aos nós de condição e ação.

Após a incorporação dos novos comportamentos, a BT passou a apresentar 32 nós e 31 ligações. A composição dessa estrutura final é apresentada na Tabela 7.

Tabela 7 – Nós da implementação final baseada na BT.

Nº	Tipo	Função / Comportamento
1	*Selector*	Raiz
2	*Sequence*	Ataque
3	*Condition*	*ForceAttack / Attacking*
4	*Action*	Executar ataque
5	*Sequence*	Busca no esconderijo
6	*Condition*	Avaliar busca em esconderijo
7	*Selector*	Lanterna / busca
8	*Sequence*	Detecção de lanterna
9	*Condition*	*CanSeePlayerLight*
10	*Action*	Forçar detecção
11	*Action*	Executar busca em esconderijo
12	*Sequence*	Perseguição
13	*Condition*	Avaliar perseguição
14	*Selector*	Decisão da perseguição
15	*Sequence*	Captura

Nº	Tipo	Função / Comportamento
16	*Condition*	Jogador em alcance de ataque
17	*Action*	Forçar ataque
18	*Sequence*	Jogador escondido
19	*Condition*	Jogador escondido próximo
20	*Selector*	Com ou sem lanterna
21	*Sequence*	Detecção de lanterna durante perseguição
22	*Condition*	*CanSeePlayerLight*
23	*Action*	Forçar detecção
24	*Action*	Iniciar busca no esconderijo
25	*Action*	Executar perseguição
26	*Sequence*	Investigação
27	*Selector*	Estímulo de investigação
28	*Condition*	*CanHearPlayer*
29	*Condition*	*CanSeePlayerLight*
30	*Condition*	Investigação ativa / temporizador
31	*Action*	Executar investigação
32	*Action*	Executar patrulhamento

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 7, a estrutura final da BT é organizada hierarquicamente a partir de um nó raiz do tipo *selector*, que direciona a avaliação dos diferentes comportamentos. Os nós *sequence* agrupam condições e ações que precisam ser executadas de forma ordenada, enquanto os nós *condition* verificam as

condições necessárias para que determinado comportamento seja executado. Os nós *action*, por sua vez, representam as operações realizadas pelo NPC.

A estrutura contempla os comportamentos de ataque, busca em esconderijos, perseguição, investigação e patrulhamento. A organização dos nós permite que comportamentos específicos sejam subdivididos em condições e ações menores.

Por exemplo, a perseguição possui estruturas destinadas à captura do jogador, à verificação de um esconderijo próximo e à detecção da lanterna durante a perseguição. Da mesma forma, o ramo de investigação reúne condições relacionadas à percepção sonora, à percepção da lanterna e ao controle da investigação.

Dessa maneira, o crescimento da BT ocorreu pela adição de novos nós e ligações dentro da estrutura hierárquica, sem a necessidade de transformar cada novo comportamento em um estado independente. Enquanto a FSM ampliou sua estrutura por meio da criação de estados e do estabelecimento de transições entre eles, a BT ampliou sua estrutura pela composição de novos nós de controle, condições e ações. Essa diferença demonstra que as duas arquiteturas absorveram o aumento de comportamentos por mecanismos estruturais distintos.

Como medida complementar à análise de escalabilidade, foi calculada a complexidade ciclomática (CC) do código-fonte da IA nas versões inicial e final de cada implementação. A CC de cada método corresponde a 1 somado ao número de pontos de decisão presentes nele, como estruturas condicionais, laços, *cases* e operadores lógicos de curto-circuito. As decisões presentes em expressões lambda, empregadas na construção da árvore, são contabilizadas no método que as contém.

Na FSM inicial, a CC de cada um dos 20 métodos é apresentada na Tabela 8.

Tabela 8 – CC por método da implementação inicial da FSM.

Nº	Método	CC
1	*Awake*	3
2	*Start*	5
3	*Update*	4
4	*SwitchState*	8

Nº	Método	CC
5	*PatrolBehavior*	4
6	*WaitingBehavior*	2
7	*WaitingRoutine*	1
8	*ChaseBehavior*	9
9	InvestigarEsconderijo	1
10	*FindNewPatrolDestination*	7
11	*StartChasing*	4
12	*AttackRoutine*	3
13	*CanSeePlayer*	5
14	*HandleTeleportCheck*	4
15	*TryTeleportNearPlayer*	4
16	*OnDrawGizmosSelected*	5
17	*OnTriggerEnter*	5
18	IniciarFadeAudioPerseguicao	1
19	RotinaFadeAudio	2
20	AtivarMonstro	1
	CC total	**78**

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 8, a FSM inicial totalizou CC de 78 distribuída em 20 métodos. Os maiores valores estão em *ChaseBehavior* (9), *SwitchState* (8) e *FindNewPatrolDestination* (7), enquanto quatro métodos apresentam CC igual a 1. Nessa versão, a lógica de decisão está distribuída entre

os métodos que implementam o comportamento de cada estado, como *PatrolBehavior* (4), *WaitingBehavior* (2) e *ChaseBehavior* (9), e nenhum método ultrapassa o valor 9.

A Tabela 9 apresenta a CC dos 26 métodos da implementação final da FSM.

Tabela 9 – CC por método da implementação final da FSM.

Nº	Método	CC
1	*Awake*	3
2	*Start*	6
3	*Update*	2
4	*OnValidate*	2
5	*OnDisable*	2
6	AtualizarAuraDebug	7
7	AtualizarPropriedadesAura	12
8	AvaliarTransicoes	36
9	ExecutarEstadoAtual	9
10	*SwitchState*	2
11	*OnExitState*	2
12	*OnEnterState*	15
13	*ExecutarGameOver*	2
14	*CanSeePlayer*	5
15	*CanHearPlayer*	3
16	*CanSeePlayerLight*	8
17	*FindNewPatrolDestination*	7

Nº	Método	CC
18	*HandleTeleportCheck*	5
19	*TryTeleportNearPlayer*	4
20	AumentarAgressividade	4
21	RecalcularDistanciasQuadradas	1
22	IniciarFadeAudioPerseguicao	1
23	RotinaFadeAudio	2
24	*OnTriggerEnter*	6
25	AtivarMonstro	1
26	*OnDrawGizmosSelected*	7
	CC total	**154**

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 9, a CC total da FSM passou de 78 para 154, e o número de métodos passou de 20 para 26. O aumento concentrou-se em poucos métodos: AvaliarTransicoes apresenta CC igual a 36, o que corresponde a cerca de 23% do total, e, somado a *OnEnterState* (15), *AtualizarPropriedadesAura* (12) e ExecutarEstadoAtual (9), responde por 72 dos 154 pontos, ou aproximadamente 47%. Esse resultado é coerente com as 17 transições da versão final, pois as condições que ligam os seis estados são avaliadas em um único método central. Em contrapartida, *SwitchState* passou de 8 para 2.

A Tabela 10 apresenta a CC dos 29 métodos da implementação inicial da BT.

Tabela 10 – CC por método da implementação inicial da BT.

Nº	Método	CC
1	*Awake*	3

Nº	Método	CC
2	*Start*	5
3	*BuildBehaviorTree*	1
4	*Update*	1
5	*IsAttacking*	1
6	*IsInvestigating*	1
7	*ShouldChase*	4
8	*IsWaitingAtPoint*	1
9	*ExecuteAttack*	1
10	*ExecuteInvestigation*	1
11	*ExecuteChase*	10
12	*ExecuteWait*	1
13	*ExecutePatrol*	4
14	*StartChasingBehavior*	6
15	*StopChasing*	1
16	*StartWaiting*	1
17	*TriggerAttack*	2
18	*WaitingRoutine*	1
19	*InvestigarEsconderijo*	1
20	*AttackRoutine*	3
21	*CanSeePlayer*	5
22	*FindNewPatrolDestination*	7

Nº	Método	CC
23	*HandleTeleportCheck*	5
24	*TryTeleportNearPlayer*	5
25	*OnTriggerEnter*	5
26	IniciarFadeAudioPerseguicao	1
27	RotinaFadeAudio	2
28	AtivarMonstro	1
29	*OnDrawGizmosSelected*	5
	CC total	**85**

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 10, os métodos da implementação inicial da BT somam CC de 85. Os maiores valores estão em *ExecuteChase* (10) e *StartChasingBehavior* (6), e 14 dos 29 métodos apresentam CC igual a 1. Os métodos *BuildBehaviorTree* e *Update* possuem CC igual a 1, o que indica que a montagem da árvore e o ciclo de decisão não concentram desvios condicionais, já que as decisões ficam nas condições e nas ações dos nós.

A Tabela 11 apresenta a CC dos 37 métodos da implementação final da BT.

Tabela 11 – CC por método da implementação final da BT.

Nº	Método	CC
1	*Awake*	3
2	*Start*	5
3	*Update*	1
4	*OnValidate*	2
5	*OnDisable*	2

Nº	Método	CC
6	*AtualizarAuraDebug*	7
7	AtualizarPropriedadesAura	12
8	ConstruirArvore	1
9	ConstruirRamoAtaque	2
10	ConstruirRamoBuscaEsconderijo	1
11	ConstruirRamoPerseguicao	5
12	ConstruirRamoInvestigacao	4
13	ConstruirRamoPatrulha	1
14	*SetBehavior*	2
15	*OnExitBehavior*	2
16	*OnEnterBehavior*	15
17	ExecutarAtaque	4
18	ExecutarBuscaEsconderijo	3
19	ExecutarPerseguicao	2
20	ExecutarInvestigacao	7
21	ExecutarPatrulha	6
22	AvaliarPerseguicao	3
23	AvaliarBuscaEsconderijo	6
24	*CanSeePlayer*	5
25	*CanHearPlayer*	3
26	*CanSeePlayerLight*	8

Nº	Método	CC
27	*FindNewPatrolDestination*	7
28	*HandleTeleportCheck*	5
29	*TryTeleportNearPlayer*	4
30	AumentarAgressividade	4
31	RecalcularDistanciasQuadradas	1
32	IniciarFadeAudioPerseguicao	1
33	RotinaFadeAudio	2
34	Executar*GameOver*	2
35	*OnTriggerEnter*	6
36	AtivarMonstro	1
37	*OnDrawGizmosSelected*	7
	CC total	**152**

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Tabela 11, a CC dos métodos da BT passou de 85 para 152, e o número de métodos passou de 29 para 37. O maior valor individual é o de *OnEnterBehavior* (15), seguido por AtualizarPropriedadesAura (12). A construção da árvore foi dividida em seis métodos (ConstruirArvore e os cinco ConstruirRamo), cuja CC varia de 1 a 5, e *Update* manteve CC igual a 1. Assim, o crescimento da BT foi distribuído entre métodos de menor complexidade, sem um método equivalente ao AvaliarTransicoes da FSM.